Save Point

Testing

AlotaNotes Quiz App · Prepared for Logan Young, Save Point Limited · 23 October 2026

Running tests

```
flutter analyze
flutter test
```

175 tests, all passing as of the last work session.

End-to-end (integration) test

```
flutter test integration_test -d linux
```

`integration_test/app_flows_test.dart` builds and launches the real app on Linux desktop and drives one full session through the UI: wrong then right sign-in, AI provider setup on the Account tab, creating a subject and a note, generating questions, editing one, bulk importing two files, taking the quiz (one right, one wrong answer), changing the default question format, exporting, then signing out and back in to check everything persisted.

What's real and what isn't:

- **Real**: the SQLite database file (FFI), `shared_preferences`, `flutter_secure_storage` (libsecret, your actual keyring), and HTTP. The AI provider is pointed, through the Account screen's base URL field, at a local `HttpServer` that speaks the OpenAI Chat Completions format, so no API key or network is needed. The test also checks the request it sent (bearer key, model, note text).
- **Faked**: `path_provider` points at a temp dir, because on Linux the app's documents directory is `~/Documents` and the test must never touch your real `quizapp.db`. The file picker and share sheet are native OS dialogs a test can't click, so they're faked at their platform interface.
- **Restored**: the API key and settings you had before the run are put back afterwards.
- **One shortcut**: new questions are first due a day after creation. The test moves their due dates into the past, through the same database file the app has open, to stand in for that day passing.

It needs the Linux desktop toolchain:

```
sudo apt install clang cmake ninja-build pkg-config libgtk-3-dev libsecret-1-dev liblzma-dev
```

Nothing here counts towards the coverage number above; `flutter test --coverage` only runs `test/`.

Coverage

Every line in `lib/` is covered (1042/1042 lines). To check:

```
flutter test --coverage
dart run tool/check_coverage.dart
```

`tool/check_coverage.dart` reads `coverage/lcov.info` directly, so it doesn't need `lcov` / `genhtml` installed. It lists any uncovered lines and exits non-zero below 100% (pass a number, e.g. `95`, to use a different threshold). It also fails if a `lib/` file never appears in the report, which happens when no test imports the file at all. Flutter's report leaves those files out entirely, so a plain percentage would hide them.

This is **line** coverage, not branch coverage. A line counts as covered once any part of it runs, so a one-line `a ? b : c` is covered even if only one side ever runs. The one place that matters here is listed under "What's not covered" below.

Structure

```
integration_test/
└── app_flows_test.dart               # one full session on Linux desktop, see above
test/
├── helpers/
│   └── test_env.dart                 # shared DB setup, plugin fakes, async polling helpers
├── core/
│   ├── scheduling_test.dart          # pure function, table-driven cases
│   ├── grading_test.dart             # pure function
│   └── text_extraction_test.dart     # hand-built .docx/.pptx, generated .pdf
├── ai/
│   ├── base_test.dart                # JSON response parsing, no network
│   ├── prompts_test.dart             # prompt templates, linked-note block
│   ├── provider_factory_test.dart
│   ├── openai_provider_test.dart     # HTTP mocked via http's MockClient
│   └── anthropic_provider_test.dart
├── auth/
│   └── auth_service_test.dart
├── settings/
│   └── app_settings_test.dart        # in-memory shared_preferences
├── storage/
│   ├── db_test.dart                  # backend selection, real on-disk open
│   └── repos_test.dart               # real in-memory SQLite via sqflite_common_ffi
├── services/
│   ├── question_generation_service_test.dart  # fake AIProvider
│   └── data_export_service_test.dart
```

```
└── screens/                          # real widget trees, tester.tap()
    ├── app_shell_test.dart           # main(), auth gate, login, RootShell tabs, Account
    ├── notes_home_screen_test.dart
    ├── folder_detail_screen_test.dart
    ├── bulk_import_staging_screen_test.dart
    ├── note_editor_screen_test.dart
    ├── question_screens_test.dart    # NoteQuestionsScreen + QuestionEditorScreen
    ├── quiz_screen_test.dart
    └── settings_screen_test.dart
```

Why widget tests, not device-input simulation

This matters enough to spell out, because it cost real time to learn during this project's first working session.

Early manual verification drove the running app on an Android emulator via `adb shell input tap <x> <y>`, the same way you'd script input from outside the app entirely. One specific interaction (tapping a floating action button) reliably failed to register, while everything else worked. Chasing it down (swapping release/debug builds, removing `IndexedStack`, replacing the button with a plain synchronous counter, comparing `top:`-anchored vs. `bottom:`-anchored `Positioned` widgets at identical screen coordinates) eventually showed the real pattern: anything positioned via `bottom:` in that particular screen had its hit-test region diverge from its paint position, while identical `top:`-anchored widgets worked every time and a plain bottom-nav tap (outside that screen entirely) also worked every time.

That's a property of how instantaneous, zero-duration synthetic ADB taps race a specific headless, software-rendered emulator's gesture-arena handling. It is not something a real finger tap on a real device would ever reliably trigger, and it is not a bug in this app's code, which was proven correct independently (see below).

`flutter_test`'s `tester.tap()` sidesteps this category of problem entirely: it drives the widget tree directly at the framework level, with no real touch/display/OS input pipeline involved, so it's both deterministic and the actual right tool for verifying this kind of interaction. `flutter test` can drive real widget trees headlessly without needing a display at all.

The practical rule this leaves behind: if you need to verify that a button/gesture does the right thing, write a widget test for it. Manually driving the running app via ADB is still useful for a final "does it look right, does it crash on startup" sanity check (see `running_locally.md`), but it is not a reliable way to verify that a specific tap does what it's supposed to.

Key testing patterns used throughout

- **Dependency injection over mocking real I/O**: every repo/service takes its dependencies (a `Database`, an `AIProvider`, an `http.Client`) via constructor, so tests pass real in-memory SQLite connections and `MockClient` instances instead of monkeypatching internals.

- **Real in-memory SQLite over hand-written fakes** for repo/widget tests (via `sqflite_common_ffi`'s `databaseFactoryFfi`), higher fidelity (catches real SQL/constraint bugs) and just as fast. Each test opens its own isolated in-memory database with `OpenDatabaseOptions(singleInstance: false)`; without that flag, sqflite caches connections by path, and every test in a file would share (and collide on) the same `:memory:` database.
- `tester.runAsync()` **for any widget test that touches real I/O.** `sqflite_common_ffi` does genuine async work (a real native/isolate round trip per query), and Flutter's default test zone (`FakeAsync`) doesn't drive that kind of real `Future` to completion on its own. Without `runAsync`, `pumpWidget` and a widget's own internal DB calls race each other and Flutter throws a "Guarded function conflict."
- **Poll for real async completion instead of** `pumpAndSettle()`**, when a** `CircularProgressIndicator` **might be on screen.** `pumpAndSettle()` refuses to work while an indeterminate spinner is present (its animation never "settles" on its own), so widget tests wait for the actual signal they care about (a specific widget's disappearance, a specific list item appearing) instead, with real delays.
- **HTTP mocked via** `http`**'s own** `testing.dart` for `ai/`: every error branch (non-2xx status, connection failure, invalid JSON, unexpected response shape) has a dedicated test case, with no real network or API key ever required.
- **Hand-built** `.docx` **/** `.pptx` **fixtures** (in `text_extraction_test.dart`, built with the `archive` package's `ZipEncoder` at test time) rather than checked-in binaries, so the exact shape of what's being parsed stays visible in the test itself. The `.pdf` fixture is generated at test time with Syncfusion's own PDF writer, the same package the app reads with.
- **Every platform plugin is faked through its official test hook**, set up per test by `setUpTestEnv()` in `test/helpers/test_env.dart`:
  - `flutter_secure_storage`: `FlutterSecureStorage.setMockInitialValues`
  - `shared_preferences`: `InMemorySharedPreferencesAsync`
  - `file_picker`: a `FakeFilePicker` whose `result` each test sets (null means the user cancelled)
  - `share_plus`: a `FakeSharePlatform` that records the shared files
  - `path_provider`: a `FakePathProvider` pointing at a fresh temp directory

  No test ever reaches a real platform channel.
- **Screens that reach** `RootShell` **run as Linux** (`variant: TargetPlatformVariant.only(TargetPlatform.linux)`). That makes the real, unmodified `openAppDatabase()` take its FFI branch and open `quizapp.db` in the fake documents directory, instead of calling the `sqflite` plugin, which has no implementation under `flutter test`. The variant resets the platform override after each test on its own.
- **Seams only where a dependency can't be faked from outside.** `NoteEditorScreen` takes an optional `providerBuilder` (so "Generate" can use a fake `AIProvider` instead of real HTTP), and `appDatabaseFactory({isWeb, platform})` is split out of `openAppDatabase()` (so the web and Android/iOS backend choices can be asserted from the VM). Both default to exactly what the app used before.
- **Navigation waits use real delays too.** `settleNavigation()` interleaves frame pumps with short real delays, because many pushes and pops sit next to real DB work (create a note, then open it; refresh a

list on return). Pumping frames alone finishes before that I/O does.

What's not covered

- **The web-only runtime path.** `kIsWeb` is a compile-time constant, so the VM test runner can never take the web side of `openAppDatabase()`'s path expression, and it can never actually open the IndexedDB factory (on the VM it throws `UnsupportedError`, which `db_test.dart` asserts). The web build is checked by building and serving it instead: `flutter build web` and the Docker image (see `hosting.md`). Neither of those exercises the app's behaviour in a real browser.
- **End-to-end on mobile.** The integration test runs on Linux desktop only. It should also run on an Android emulator or phone (`-d <device>`), but that hasn't been tried, and it would write the API key and settings into that device's real stores rather than a test copy. The ADB-tap approach was tried and found unreliable (see above).
- **Native dialogs.** The file picker and share sheet are faked even in the integration test, so opening and dismissing the real OS dialogs is still only checked by hand.
- **The integration test hasn't been run yet.** It was written on a machine without the Linux desktop toolchain. Run it once the packages above are installed, and remove this line when it passes.